

最短路-Dijkstra-板子

Dijkstra 单源最短路（无向带权图）板子

```
#include<bits/stdc++.h>
//#define int long long
using namespace std;
using ll=long long;
const ll inf=1ll<<60;
```

//这段代码实现了经典的 **Dijkstra** 单源最短路（无向带权图），
//用优先队列按距离扩展节点，最终输出从 **s** 到 **t** 的最短距离
//（如果不可达则输出 **inf** 的数值）

```
void work(){
    int n,m,s,t;
    cin>>n>>m>>s>>t;    s--;
    t--;
    vector<vector<pair<int,int> > > g(n);
    for(int i=0;i<m;i++){
        int u,v,w;
        cin>>u>>v>>w;
        u--;
        v--;
        g[u].emplace_back(v,w);
        g[v].emplace_back(u,w); //无向图
    }
    //仍用默认 max-heap，但把距离存成负数：q.emplace(-dist, node);    //这样 t
    priority_queue<pair<ll,int> > q;    //存点，边权
    vector<ll> dis(n,inf);    //到达每个点时的最短路
    dis[s]=0;    //起点为零
    q.emplace(0,s);
    while(size(q)){
        auto [d,u]=q.top();    //存 距离负值-dist，节点u
        q.pop();
        if(-d != dis[u]) continue;
        for(auto [v,w] : g[u]){
            if(dis[u]+w<dis[v]){    //如果s->u->v路径长度比原来要小，则更新，那么就
                dis[v]=dis[u]+w;
                q.emplace(-dis[v],v);
            }
        }
    }
    cout<<dis[t];    //到达终点的 最短路径
} signed main(){
    ios::sync_with_stdio(0);
    cin.tie(0);
```

```
        int t=1;
//      cin>>t;
        while(t--){
            work();
        }
    }

    /*
    4 6 1
    1 2 2
    2 3 2
    2 4 1
    1 3 5
    3 4 3
    1 4 4

    0 2 4 3
    */
```